Process Rewards for Mathematical Reasoning: GRPO Baselines and Failure Analysis

Hongyi Ding **Yang Zeng** **Jasmine Jiang**

Abstract

We compare outcome rewards with a mixture of outcome and process rewards for GRPO fine-tuning of Qwen2.5-Math-1.5B. With about six million generated training tokens per run, outcome-only training reaches 54.00% on MATH500 and 67.02% on GSM8K. Adding a frozen 7B process reward model (PRM) gives 53.80% and 64.97%, respectively. Both paired confidence intervals include zero. We examine the saved responses to understand why the extra signal did not help. The PRM misses some clear errors, and group normalization can give wrong responses positive advantages even when their process scores are low. We also describe C-bounded, a preliminary follow-up that limits the process signal to a bounded correction among answer-correct responses.

1 Introduction

Mathematics is a useful setting for reinforcement learning because many final answers can be checked automatically. However, a correct answer tells us little about the quality of the derivation. A PRM could provide additional feedback by scoring the intermediate steps. We ask whether this feedback helps a small model learn to solve problems more accurately.

A verifier or PRM supplies rewards, while GRPO uses those rewards to update the policy. We keep GRPO fixed and compare the two reward choices. At evaluation, the model produces one greedy answer per problem, without PRM reranking. Our main measure is answer accuracy; checking the reasoning itself requires a separate assessment.

Our original proposal focused on comparing PRM adaptation using both successful and failed responses with adaptation using failures alone. When direct PRM mixing did not improve the baseline, we first checked the implementation and examined the training signal. This report focuses on that comparison and analysis. The original adaptation comparison is still incomplete.

2 Related work

GRPO estimates advantages from groups of responses without training a critic (Shao et al., 2024). We use LoRA to update the policy while keeping the base weights fixed (Hu et al., 2021).

Lightman et al. (2023) compared outcome and process supervision and released human step labels. Qwen's PRM study discusses problems with label reliability and with evaluating process models through final-answer accuracy alone (Zhang et al., 2025). These issues matter here: a high process score may not correspond to correct mathematics.

PURE uses min-form credit assignment at the step level (Cheng et al., 2025). Our baseline takes the minimum step score as a response reward and then applies standard GRPO, so the two methods differ in how they assign credit. Our work so far is mainly empirical: we compare the rewards, check the resulting advantages, and study failure cases. C-bounded is a small follow-up to these findings; outcome gating itself is an existing idea.

ECE285: Deep Generative Models. Midterm project report, November 9, 2026.

3 Methods

3.1 Outcome and process rewards

For each problem q, the policy π_θ samples $G = 8$ chain-of-thought responses $o_1, \ldots, o_G$. We set $v_i = 1$ when response i terminates normally and its answer can be parsed under the fixed output protocol. The outcome reward $y_i \in \{0, 1\}$ is one when the final boxed answer matches the reference under our equivalence checker. Truncated or unparseable responses receive zero.

For responses with $v_i = 1$, the frozen PRM scores steps separated by double line breaks. Following the Qwen interface, we insert a marker after each step and read the positive-class probability (Zhang et al., 2025). The response score is

$$\widetilde{m}_i = \begin{cases} \min_{1 \le k \le K_i} p_\phi(\text{valid} \mid q, s_{i,\le k}), & v_i = 1, \\ 0, & v_i = 0, \end{cases} \qquad 0 \le \widetilde{m}_i \le 1. \tag{1}$$

The PRM is not called for invalid responses; their zero score is a placeholder in the code. We compare outcome-only training (A) with direct reward mixing (B):

$$R_i^A = y_i, \qquad R_i^B = y_i + \lambda \widetilde{m}_i, \qquad \lambda = 0.2. \tag{2}$$

A complete, parseable wrong answer can therefore receive a positive process term in B. The PRM stays frozen during training.

3.2 From rewards to parameter updates

Within each group of responses to the same question, GRPO computes

$$\bar{R} = \frac{1}{G} \sum_j R_j, \quad s_R = \sqrt{\frac{1}{G-1} \sum_j (R_j - \bar{R})^2}, \quad \widehat{A}_i = \frac{R_i - \bar{R}}{s_R + \delta}, \qquad \delta = 10^{-4}. \tag{3}$$

Each response gets one scalar advantage, shared by its active tokens and treated as a constant during backpropagation. We use `loss_type="grpo"`, which averages over tokens within each response (Shao et al., 2024; Hugging Face, n.d.). With $h_{i,t} = (q, o_{i,<t})$, define

$$\begin{aligned} \rho_{i,t}(\theta) &= \frac{\pi_\theta(o_{i,t} \mid h_{i,t})}{\pi_{\text{old}}(o_{i,t} \mid h_{i,t})}, \qquad u_{i,t} = \frac{\pi_{\text{ref}}(o_{i,t} \mid h_{i,t})}{\pi_\theta(o_{i,t} \mid h_{i,t})}, \\ \kappa_{i,t}(\theta) &= u_{i,t} - \log u_{i,t} - 1. \end{aligned} \tag{4}$$

The old-policy denominator is detached, and π_{ref} is a fixed reference model. The loss for a group is

$$\mathcal{L}(\theta) = -\frac{1}{G} \sum_{i=1}^{G} \frac{1}{T_i} \sum_{t=1}^{T_i} \left[\min\Big\{ \rho_{i,t}\, \text{sg}(\widehat{A}_i), \text{clip}(\rho_{i,t}, 1-\epsilon, 1+\epsilon)\, \text{sg}(\widehat{A}_i) \Big\} - \beta \kappa_{i,t} \right], \tag{5}$$

where $\epsilon = 0.2$, $\beta = 0.04$, and T_i is the number of active completion tokens. Here sg denotes stop-gradient. These equations describe the loss selected in our runs.

We make one optimizer iteration per rollout batch. If old log probabilities are not cached, TRL uses detached current log probabilities as the denominator. The ratio is then one at the current parameters, but its derivative is still nonzero. Locally, ignoring clipping and KL, response i contributes $-\widehat{A}_i \sum_t \nabla_\theta \log \pi_\theta(o_{i,t} \mid h_{i,t})/T_i$ to the loss gradient. A positive advantage pushes up the response's log probability; a negative one pushes it down. The reward itself need not be differentiable. Only the policy's LoRA parameters are updated.

4 Experiments

4.1 Models, data, and controls

We train the *base* Qwen2.5-Math-1.5B model (Yang et al., 2024) with rank-16 LoRA, alpha 32, and no dropout. B uses Qwen2.5-Math-PRM-7B without further adaptation. The implementation uses TRL 0.23.1 and vLLM on one A100 80 GB. We train on 6,993 prepared MATH problems and hold out 500 for validation. We test on MATH500 (500 problems) and GSM8K (1,319 problems) (Hendrycks et al., 2021; Cobbe et al., 2021). GSM8K is used only for evaluation, to check transfer beyond the training dataset.

Table 1: Shared A/B training settings. Both runs use the same actor initialization, prompt template, and verifier.

Setting	Value	Setting	Value
Training seed	17	Learning rate	5×10^{-6}, constant
Prompts/update	4	Responses/prompt	8
Sampling	temperature 1, top-p 1	Output cap	2,048 tokens
Optimizer	AdamW; grad clip 1	Iterations/batch	1
Policy loss	per-response GRPO	KL / ratio clip	0.04 / 0.2
Policy LoRA	rank 16, alpha 32	B process weight	0.2

Before this comparison, we fixed percentage-unit verification and the handling of unit-aware answers. We also normalized CRLF line endings when preparing PRM inputs, without changing the actor's text. Both runs use the corrected code. The results below exclude earlier runs with these implementation errors.

Each run stops after the first complete update reaching six million generated completion tokens, including failed and truncated outputs. A uses 6,022,435 tokens over 284 updates; B uses 6,008,457 over 280 updates. They start from the same adapter and follow the same question order for the first 1,120 groups. A processes 16 additional groups. Matching generation tokens therefore gives slightly different update counts, and B also has the extra cost of PRM scoring.

4.2 Final-checkpoint evaluation

We evaluate both final checkpoints with greedy decoding, a 2,048-token cap, and the same verifier. There is no voting or PRM reranking. A separate verification script rechecks all 3,638 saved predictions. We estimate confidence intervals for B$-$A with 10,000 paired bootstrap samples over questions (seed 285).

Table 2: Outcome-only GRPO versus direct PRM mixing at about 6M training tokens. Differences and intervals are in percentage points (pp). Both runs use seed 17.

Test	A: outcome	B: mixed	B$-$A	Paired 95% interval
MATH500	54.00%	53.80%	-0.20	$[-4.00, +3.60]$
GSM8K	67.02%	64.97%	-2.05	$[-4.25, +0.08]$

B nearly matches A on MATH500 and falls behind by 2.05 points on GSM8K. Both confidence intervals include zero, so this single-seed comparison does not give clear evidence of a difference. It also cannot establish that the methods are equivalent. The intervals account for variation across questions; they do not include variation across training seeds or repeated evaluations.

Training cost. B processes another 5,281,779 tokens through the PRM (7,724,200 with padding). Scoring takes 593 seconds, including preprocessing and transfer. The recorded training times are 2.70 h for A and 2.22 h for B, with peak allocated memory of 31.60 and 44.88 GiB. These are shared-GPU measurements: changes in GPU load and response length prevent a clean speed comparison. The token budget above covers actor generation, so it does not account for all of B's additional compute.

5 Failure analysis

5.1 Implementation checks

We checked model versions, settings, and checkpoints, then replayed the reward calculation from saved responses. For all 7,304 scored B responses, text hashes and step spans match the PRM inputs. The advantages also match the values passed into the policy loss, with no second normalization.

5.2 Effect of group normalization

A process weight of 0.2 does not limit the final advantage to that scale. When all responses in a group have the same outcome reward, Eq. (2) gives

$$\widehat{A}_i^B = \frac{\widetilde{m}_i - \overline{\widetilde{m}}}{s_{\widetilde{m}} + \delta/\lambda} = \frac{\widetilde{m}_i - \overline{\widetilde{m}}}{s_{\widetilde{m}} + 5 \times 10^{-4}}. \tag{6}$$

The factor 0.2 cancels except in the stabilizer. A has zero reward advantages in these groups. B can still produce substantial positive and negative advantages, including when every final answer is wrong. This behavior follows from ordinary group normalization. The KL term remains active in both runs.

In B, 242 of 1,120 groups (21.61%) are all-correct or all-wrong. They account for 19.92% of the absolute advantage sum, not gradient norm. There are 577 wrong-answer responses with positive advantages: 571 in all-wrong groups and six in mixed groups. Their rewards exceed the group average, which alone says nothing about their mathematical progress.

Table 3: Two selected training responses. PRM scores and actual advantages can tell different stories.

Observed failure	Min PRM	Advantage	Interpretation
$4 \cdot 6 + 8 \cdot 3 - 28/2$ reported as 36 rather than 34	0.9797	-1.072	PRM miss, but the response is still penalized.
Nine-node coloring: invalid index 9 and inconsistent reported counts; correct count 54	0.0538	$+1.070$	All eight answers are wrong; relative scaling rewards a low-scored response.

The first response claims its code returns 36, although it returns 34. Despite the high PRM score, training penalizes this response. In the second case, enumeration of the original graph gives 54 colorings. The response claims both 132 and 12 and uses an invalid node index, yet receives positive advantage within its all-wrong group.

Among the ten highest-PRM responses with outcome reward zero, four contain mathematical errors, four have boxing or unit mismatches, and two involve non-unique or equivalent answers that the checker does not cover. Only one has positive advantage. A scoring error therefore needs to be considered alongside the resulting training signal.

5.3 Answer formatting and other explanations

On GSM8K, B has fewer terminated but unparseable outputs than A (53 versus 89). It has more parsed wrong answers (368 versus 314) and more truncations (41 versus 32). Missing answers alone therefore do not explain the lower accuracy.

Some responses leave a wrong value in the answer box after deriving or accepting the correct value later. An assistant identified this pattern in 31 of 44 selected A-only-correct cases and six of seven reverse cases. These are post-hoc reviews rather than human gold labels; official accuracy is unchanged. PRM scores do not rise monotonically, and average step counts remain similar (11.917/11.899). The evidence shows scoring and formatting failures, but does not establish reward hacking or explain how much each issue contributes to the test gap.

6 Preliminary follow-up: C-bounded

Based on this analysis, we tried adding a small process correction after computing outcome advantages, rather than mixing the rewards before normalization. Let $I_+ = \{j : y_j = 1\}$ and let $\bar{m}_+$ be the mean PRM minimum among answer-correct responses. We define

$$\widehat{A}_i^{\mathrm{CB}} = \underbrace{\frac{y_i - \bar{y}}{s_y + \delta}}_{\widehat{A}_i^{\mathrm{out}}} + c_i, \qquad c_i = 0.2\, y_i\, (\widetilde{m}_i - \bar{m}_+). \tag{7}$$

The correction is zero when I_+ is empty, and we do not normalize again after adding it. Since scores lie in $[0, 1]$, $|c_i| \le 0.2$ and $\sum_i c_i = 0$. Wrong answers receive no process correction and cannot have positive reward advantages; all-wrong groups have zero reward advantages. KL can still produce a gradient. This construction limits the process signal, but also gives up any useful feedback the PRM might provide on failed responses.

Table 4: Preliminary A/C-bounded results from a separate experiment. Each run uses 288 updates and about 6.1–6.3M tokens.

Seed	MATH500: A $\to$ C	GSM8K: A $\to$ C	Wrong positive advantages
17	$53.00 \to 57.80\%$	$66.87 \to 69.37\%$	0 in both arms
42	$52.80 \to 54.80\%$	$67.48 \to 67.93\%$	0 in both arms

Here we use a separately adapted 7B scorer, frozen during RL. The adaptation data contain four responses for each of 512 problems and 2,048 shared step-label anchors. This differs from B's unadapted scorer, so Table 4 should be read separately from Table 2. Both seeds improve, although seed 42 has wide paired intervals: $[-1.00, +5.00]$ pp on MATH500 and $[-1.52, +2.43]$ pp on GSM8K. The results are encouraging but do not yet establish a stable gain or separate the effect of adaptation from that of the advantage correction.

7 Limitations and next steps

Evaluation. The main comparison has one training seed and the follow-up has two. MATH500 is small, and we have already inspected the test results, so further design decisions need a separate validation set. Answer accuracy also does not measure process validity: our checker can reject equivalent answers, and a correct answer can follow an invalid derivation. Step splitting and minimum aggregation may make PRM scores sensitive to formatting. Exact-overlap checks cannot rule out pretraining exposure or paraphrases. The PRM training corpus also overlaps our validation questions.

Training and attribution. Our runs use single-card LoRA and a specific sampling setup. vLLM sampling correction is disabled; the reward checks do not rule out numerical differences between sampling and learning. The preliminary seed-17 runs also have different restart histories, while seed 42 has matched restart points. C-bounded changes gating, signal scale, and normalization together. To learn which part matters, we need controls with matched signal magnitude and with shuffled process rankings. A representative, independent review is also needed before claiming that the derivations have improved.

Plan for the final report. We will first check whether key evaluations repeat and arrange an independent review of reasoning steps. If resources permit, we will then run the component controls above and finish the error and cost analysis. The larger-budget continuation is paused, with no jointly evaluated endpoint to report. Code and existing results are available in the project repository.[1]

Direct PRM mixing has not improved this baseline so far. Group normalization can turn a small process reward into a strong training signal. C-bounded limits that signal, but its effect on reasoning needs further study.

[1] `https://github.com/mafeifei798/ece285-math-rl`

References

Cheng, J., Xiong, G., Qiao, R., et al. (2025). Stop Summation: Min-Form Credit Assignment Is All Process Reward Model Needs for Reasoning. *NeurIPS 2025*. arXiv:2504.15275v3.

Cobbe, K., Kosaraju, V., Bavarian, M., et al. (2021). Training Verifiers to Solve Math Word Problems. arXiv:2110.14168v2.

Hendrycks, D., Burns, C., Kadavath, S., et al. (2021). Measuring Mathematical Problem Solving With the MATH Dataset. arXiv:2103.03874v2.

Hu, E. J., Shen, Y., Wallis, P., et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685v2.

Hugging Face (n.d.). TRL v0.23.1: GRPO Trainer. Versioned software documentation. Accessed November 9, 2026.

Lightman, H., Kosaraju, V., Burda, Y., et al. (2023). Let's Verify Step by Step. arXiv:2305.20050.

Shao, Z., Wang, P., Zhu, Q., et al. (2024). DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv:2402.03300v3.

Yang, A., Zhang, B., Hui, B., et al. (2024). Qwen2.5-Math Technical Report: Toward Mathematical Expert Model via Self-Improvement. arXiv:2409.12122.

Zhang, Z., Zheng, C., Wu, Y., et al. (2025). The Lessons of Developing Process Reward Models in Mathematical Reasoning. arXiv:2501.07301v2. Official PRM model card and scoring interface.